

1.LLM이란 무엇인가

LLM이란 무엇인가

LLM(Large Language Model)은 한국어로 대규모 언어 모델이라고 부릅니다. 쉽게 말하면 문장을 입력받고 문장으로 답하는 AI입니다.

예시: LLM에게 물어보면?

입력: "ALM-TEMP-402 알람이 반복 발생했습니다"

출력: 가능한 원인 후보나 확인 항목을 문장으로 정리해 줍니다.

중요한 한계

LLM이 실제 공장 로그를 직접 열어 본 것은 아닙니다. 말씀씨가 좋은 전문가처럼 설명은 잘하지만, 지금 현장의 최신 상태표를 손에 들고 있는 것은 아닙니다.



한 줄 정리: LLM은 문장을 이해하고 문장으로 답하는 AI입니다. 단, 현장의 최신 데이터는 스스로 볼 수 없습니다.

LLM이 잘하는 일과 어려운 일

✓ LLM이 잘하는 일

- 긴 문서를 짧게 요약
- 어려운 내용을 쉬운 말로 설명
- 내용 분류 및 정리
- 문서 초안 작성
- 알람 로그를 사람이 읽기 쉽게 정리
- 매뉴얼 기반 확인 항목을 문장으로 작성

✗ LLM이 어려운 일

- 회사 DB를 스스로 조회하는 것
- 오늘 발생한 최신 알람을 자동 확인하는 것
- 실시간 현장 데이터에 접근하는 것

⚠ LLM은 똑똑한 보고서 작성자에 가깝지만, 자료 창고를 직접 뒤지는 담당자는 아닙니다.

제조 업무에서 LLM의 한계

제조 업무에서는 단순한 문장 답변만으로 충분하지 않습니다.

다음 사항들을 반드시 확인해야 합니다.

어떤 설비?

어떤 설비에서 알람이 났는지 확인 필요

반복 여부?

같은 알람이 반복되었는지 확인 필요

매뉴얼 조치?

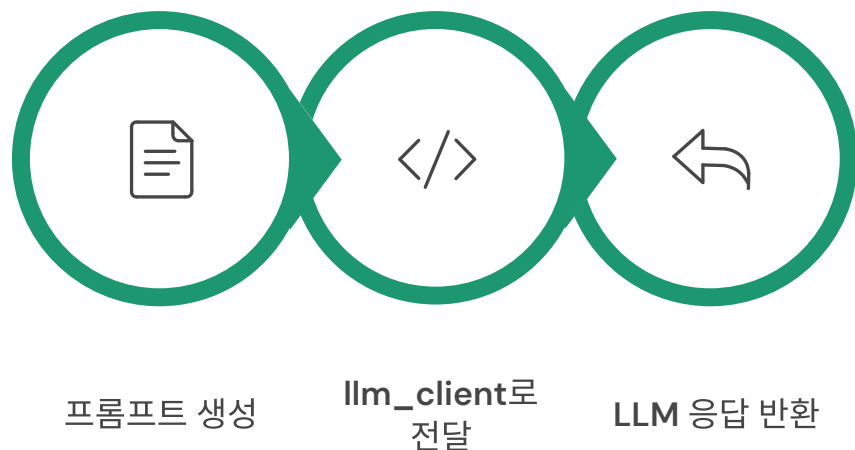
매뉴얼에 어떤 조치가 있는지 확인 필요

⊗ ⚠ LLM에게 정보 없이 "원인이 뭐야?"라고 물으면 **일반적인 추측**을 할 수 있습니다. 제조 현장에서는 이런 추측이 위험할 수 있습니다.

그래서 교육용 실습에서는 **LINE-07, EQP-EV-03, ALM-TEMP-402** 같은 가상 정보를 사용해, LLM이 근거를 바탕으로 답하도록 만드는 구조를 배웁니다.

실제 LLM 호출 구조

실제 LLM 호출은 사용자의 질문을 LLM 서비스에 보내고 응답을 받는 흐름입니다.



왜 `llm_client.py`를 따로 분리할까요?

예제 파일들은 `llm_client.py`의 `generate_response(prompt)`를 통해 LLM 호출부를 분리합니다. 이렇게 하면 Day1 실습 파일은 **프롬프트 생성과 실행 흐름 이해**에 집중할 수 있습니다.



비유:

실습 파일은 **주문서를 작성**하고, `llm_client.py`는 그 주문서를 실제 요리사에게 전달하는 창구입니다.

프롬프트 입력과 응답 생성 흐름

LLM은 사용자의 마음을 읽지 않습니다. 입력된 프롬프트를 보고 응답을 생성합니다.

❌ 모호한 프롬프트 (나쁜 예)

⊗ "이거 왜 이래?"

LLM이 무엇을 물어보는지 알 수 없어 모호한 답변이 나옵니다.

✅ 구체적인 프롬프트 (좋은 예)

⊙ "EQP-EV-03에서 ALM-TEMP-402 알람이 반복 발생했습니다. 로그와 매뉴얼 근거로 원인 후보와 확인 항목을 정리해 주세요."

📌  **비유:** 프롬프트는 택배 주소와 비슷합니다. 주소가 대충 적혀 있으면 택배가 엉뚱한 곳으로 갈 수 있듯이, 프롬프트가 모호하면 LLM 응답도 모호해질 수 있습니다.

LLM이 회사 DB/최신 로그를 조회하지 못하는 이유

LLM은 기본적으로 회사 내부 시스템과 자동으로 연결되어 있지 않습니다.

회사 내부에 따로 저장된 정보들

- 사내 DB
- 최신 알람 로그
- 내부 매뉴얼
- 품질 지표

LLM이 이 정보를 보려면?

- 누군가 데이터를 직접 제공하거나
- Agent가 검색 도구나 DB 조회 도구를 연결해야 합니다



비유: 상담원이 친절하게 설명할 수는 있지만, **재고 시스템을 열어 보지 않으면** 현재 재고 수량을 정확히 말할 수 없습니다. LLM도 최신 업무 데이터는 **따로 전 달받아야** 합니다.

LLM 호출과 Agent의 차이

LLM 호출

한 번의 질문에 대해 답변을 생성하는 방식

예시: "알람 원인 알려줘"라는 프롬프트를 보내고 답을 받는 것

Agent (업무 흐름 수행)

1. 설비 ID와 알람 코드 확인
2. 로그 조회
3. 매뉴얼 참고
4. 프롬프트 생성
5. LLM 응답 생성



비유: LLM은 문장을 만드는 역할에 가깝고, Agent는 자료를 모으고 순서를 정해 업무를 처리하는 담당자에 가깝습니다.

핵심 정리

1

LLM은 문장 AI

LLM은 문장을 이해하고 문장으로 답하는 AI입니다.

2

LLM의 한계

LLM은 회사 DB나 최신 알람 로그를 스스로 조회하지 못합니다.

3

Agent = LLM + 업무 흐름

Agent는 LLM에 업무 흐름을 붙인 구조입니다. 질문 분석 → 정보 탐색 → 단계 선택을 수행합니다.



🎯 다음 단계 예고: Prompt, Chain, Graph를 이해하면 2일차 RAG와 3일차 Tool 연결을 훨씬 쉽게 따라갈 수 있습니다.

4.LangChain 이 필요한 이유

LangChain이 필요한 이유

LLM을 업무에 활용하려면 단순히 질문을 보내고 답변을 받는 것만으로는 부족합니다. 실제 업무에서는 여러 단계가 필요합니다.

1

프롬프트 생성

업무에 맞는 지시문을 만듭니다

2

문서 검색

필요한 문서나 로그를 찾습니다

3

도구 호출

외부 기능이나 DB를 호출합니다

4

결과 정리

응답을 정리하고 전달합니다



 **LangChain**은 이런 LLM 기반 프로그램을 만들 때 자주 사용하는 **프레임워크**입니다. 프레임워크는 자주 쓰는 구조를 미리 준비해 둔 **도구 상자**라고 생각하면 됩니다. 1일차에는 LangChain 문법을 깊게 배우지 않고, 왜 **Prompt**와 **Chain** 같은 구조가 필요한지 이해하는 데 집중합니다.

LangChain의 Prompt, Chain, Tool, Agent 연결 개념



Prompt (프롬프트)

LLM에게 주는 작업 지시문입니다.

비유: 작업 지시서



Tool (툴)

외부 기능을 호출하는 도구입니다.

비유: 전화기나 검색기 같은 업무 도구



Chain (체인)

정해진 순서대로 실행되는 흐름입니다.

비유: 작업 순서표



Agent (에이전트)

상황에 따라 필요한 작업을 선택하는 구조입니다.

비유: 어떤 순서로 어떤 도구를 쓸지 판단하는 담당자



🎯 1일차 목표: Prompt와 Chain을 먼저 실습합니다. Tool과 Agent는 이후 일차에서 단계적으로 배웁니다.

LangGraph의 Node, Edge, State 개념

LangGraph는 Agent의 작업 흐름을 **Node, Edge, State**로 표현하는 도구입니다.

세 가지 핵심 개념

- **Node (노드):** 하나의 작업 단계
예: "알람 코드 확인", "로그 조회", "응답 생성"
- **Edge (엣지):** Node와 Node를 연결하는 길
- **State (상태):** Agent가 처리 중 기억해야 하는 정보
예: 사용자 질문, 설비 ID, 알람 코드, 조회 결과

내비게이션 비유

LangGraph 개념	내비게이션 비유
Node	경유지
Edge	도로
State	현재 목적지와 이동 정보

1일차에서는 mini graph로 단순화

1일차에서는 실제 LangChain과 LangGraph 문법을 깊게 다루지 않습니다. 처음부터 프레임워크 문법을 모두 배우면 Agent의 핵심 흐름보다 도구 사용법에 집중하게 될 수 있기 때문입니다.



📁 1일차에는 `src/day1/mini_graph_runner.py`를 사용해 Graph 개념만 단순하게 체험합니다.

mini graph의 핵심 흐름

알람 코드 있음

→ 로그 조회 단계로 이동

알람 코드 없음

→ 추가 질문 단계로 이동



🎯 1일차 최종 목표: LangGraph 전문가가 되는 것이 아니라, 조건에 따라 Agent의 실행 경로가 달라질 수 있음을 이해하는 것입니다.1.